

ANEESH THODUPUNOORI

Game Developer / Gameplay Architect

Gameplay programmer with six years of experience building the systems players see, the multiplayer they rely on, and the architecture that keeps teams shipping. Experienced in delivering scalable gameplay features, solving performance challenges, and creating maintainable foundations for long-term development.

+91 75699 14141
thodupunoorianeesh@gmail.com
Bangalore, India
[linkedin.com/in/aneesh-thodupunoori](https://www.linkedin.com/in/aneesh-thodupunoori)

TECHNICAL STACK

Primary	C++ · Unreal Engine 4/5 · C# · Unity · Blueprints · Object-Oriented Programming · Gameplay Ability System (GAS) · Multiplayer Replication
Build & Tools	Unreal Build Tool (UBT) · Custom Modules & Plugins (.uplugin / .uproject) · Unreal Insights · Debugging & Profiling · Visual Studio Debugger
Animation	Animation Blueprints · Control Rig · Blend Shapes · IK / FK · Skeletal Retargeting · Blend Spaces · Morph Targets
Gameplay	Behaviour Trees · EQS · AI Perception · Combat Systems · Vehicle Physics · Inventory Systems · UMG / UI Systems
Rendering	Material/Shader Authoring · Texturing · Lighting · Performance Profiling · Mobile Optimisation
Math	Vector Math (Dot / Cross Product) · Quaternions & Rotations · Linear Interpolation · Collision & Raycasting Math · Trigonometry for Gameplay
Concurrency	Multithreading in Unreal (Async Tasks, FRunnable) · Multithreading in Unity (Jobs System, async/await, Coroutines)
Design Patterns	Observer · Singleton · Object Pooling · Factory · State Machines · Component-based Architecture
Other	OpenGL · RayLib · Python · Lua · Git · Perforce · Mobile Development (Android / iOS)

EXPERIENCE

GamaSome · Unreal C++ Developer & Team Lead

Feb 2024 – Present | Bangalore

Led a 4-person team delivering two Unreal C++ projects: a cross-platform live metaverse game and a real-time digital human avatar system. Acted as gameplay architect for both — defining the core systems, module structure, and technical direction the rest of the team built on, alongside owning multiplayer architecture and the build/plugin setup.

Shib Eternity — Cross-platform Metaverse (Mobile · iOS · Windows)

- ▶ Designed and shipped a Fortnite-scale live multiplayer metaverse — a persistent world running multiple game modes at once — across Android, iOS, and Windows simultaneously.
- ▶ Set up and maintained custom Unreal modules and plugins for platform-specific build configurations — isolated platform code, reusable gameplay modules, and a single build pipeline producing Android, iOS, and Windows builds from one codebase.
- ▶ Built the core multiplayer architecture — server-authoritative gameplay, smooth client-side prediction, and efficient network data syncing to keep the experience responsive on lower-end mobile devices.

- ▶ Used async tasks and multithreading to keep asset streaming and background loading off the main thread, preventing hitches during gameplay across all three platforms.
- ▶ Profiled and debugged frame stability issues using Unreal's built-in performance tools, identifying bottlenecks across three different hardware classes and resolving platform-specific performance problems.
- ▶ Led sprint planning, code reviews, and day-to-day coordination across art, rigging, and engineering teams.

Digital Human Avatar System — *Real-time Conversational Characters*

- ▶ Architected a modular animation pipeline using blend shapes, IK/FK rigging, and skeletal retargeting — enabling dynamic blending of facial, head, and body behaviours for conversational characters.
- ▶ Built a parameter-based animation mapping system linking live data inputs to skeletal controls — similar to morph target logic but extended to full-body motion.
- ▶ Integrated live API triggers into the animation system, transitioning characters from static pre-animated assets to fully responsive digital humans.
- ▶ Optimised shader and rendering workflows — debugged frame instability during heavy animation playback and improved performance across different hardware.
- ▶ Standardised animation import, retargeting, and validation workflows across art, rigging, and design teams — reduced integration breakage and iteration friction.

Anarchy · Game Developer

Sep 2022 – Dec 2023 | Bangalore

Anarchy Warzone — *a full-scale battle royale built in Unity (C#). Responsible for all vehicle systems, the multiplayer inventory system, and networked session management.*

- ▶ Built all vehicle types from scratch in C# — custom wheel physics, suspension simulation, torque curves per vehicle class, and smooth networked movement using compressed transform syncing.
- ▶ Implemented a full multiplayer inventory system using the Observer pattern — UI and gameplay systems subscribed to inventory change events instead of polling, keeping pickups, equips, and drops in sync across systems with server-authoritative validation.
- ▶ Used object pooling for frequently spawned items and loot drops, and a factory pattern for vehicle and weapon spawning, to avoid runtime allocation spikes during heavy combat moments.
- ▶ Designed the battle royale session flow — lobby, map load, in-game, results — with server-controlled zone shrink logic and conflict-safe kill confirmation to prevent desync between clients.
- ▶ Used multithreading for background tasks such as loot table generation and map streaming, keeping the main thread free for gameplay during high player-count matches.
- ▶ Debugged and profiled server load under concurrent player sessions, optimising network data flow to reduce unnecessary traffic and improve stability.
- ▶ Owned the cross-platform build pipeline, delivering consistent gameplay and stable performance across different device targets.

Gods & Heroes · Game Developer

Jul 2021 – Jul 2022 | Hyderabad

Heroes of District — *a competitive 5v5 multiplayer shooter (CS:GO / Valorant style) on mobile. Owned weapon systems, inventory, custom shader work, and mobile performance optimisation.*

- ▶ Designed a flexible, component-based weapon system supporting multiple weapon types, with fire/reload logic and damage handling built for fast, fair competitive gameplay.
- ▶ Built a clean inventory and loadout system — item definitions, stats, and rarity — with UI that updates instantly as players pick up or equip gear.
- ▶ Authored custom shaders for rim lighting, surface detail, and weapon glow effects — kept lightweight enough to run smoothly on mobile GPUs.

- ▶ Profiled and debugged mobile performance — reduced draw calls, optimised texture streaming and level-of-detail settings to hit stable frame rates on mid-range Android devices during fast-paced 5v5 matches.

Resurgence Solutions · Game Developer Intern

Dec 2020 – Jun 2021 | Remote

Six-month internship building hypercasual mobile games in Unity (C#), owning gameplay programming end-to-end as the sole developer on two shipped titles.

Tap Runner — Endless Runner (Mobile · Unity)

- ▶ Built the entire endless-runner gameplay loop from scratch — lane-based procedural track generation, player movement and controls, coin collection, and enemy/obstacle spawning.
- ▶ Programmed enemy encounters and kill mechanics along the track, tuning spawn rate and difficulty ramp as the run progressed.
- ▶ Implemented the full session flow — start, run, game over, and restart — along with scoring and coin economy, as sole gameplay programmer on the title.

Hero Survivors — Round-based Melee Combat (Mobile · Unity)

- ▶ Built a round-based top-down melee combat system from scratch — player movement, melee attack and hit detection, and per-round enemy behaviour.
- ▶ Implemented round progression and win/lose conditions, where surviving consecutive rounds determined the winner.
- ▶ Owned all gameplay programming independently end-to-end, from prototype through final playable build.

PROJECTS

KingdomColliders — Clash Royale-style Card Battle Game · Solo Dev · Mobile (Unreal C++)

Built entirely alone from scratch — card system, unit AI, tower destruction, elixir economy, collision-driven combat, and mobile UI. Full ownership of project structure, modules, and build configuration.

- ▶ Built a data-driven card system with deck validation and an in-match hand/elixir management loop.
- ▶ Programmed unit AI using behaviour trees, state machines, and pathfinding — units switch between idle, march, attack, and death states; detect threats and engage on range trigger using vector math for distance and direction checks.
- ▶ Used a singleton-based game manager to coordinate match state, and a state machine to drive each unit's behaviour cleanly without scattered conditional logic.
- ▶ Debugged and optimised runtime performance for mobile constraints — maintained stable frame rates during physics-heavy collision interactions.
- ▶ Structured the project using modular, reusable design patterns — independently managed the full project setup, build configuration, and packaging pipeline.

EDUCATION

Back Stage Pass Institute of Gaming and Technology

Hyderabad · 2020

B.Tech — Computer Science & Game Development